

```

%% 1번
clear; clc;

x = [21, 22, NaN, 23, 24, 120, 25, NaN, 26, 27];

idx_missing = isnan(x);

mu0 = mean(x, 'omitnan');
sig0 = std(x, 'omitnan');

idx_outlier = abs(x - mu0) > 3*sig0;

x_temp = x;
x_temp(idx_missing | idx_outlier) = NaN;

tempMean = mean(x_temp, 'omitnan');
x_clean = x_temp;
x_clean(isnan(x_clean)) = tempMean;

mu_clean = mean(x_clean);
sig_clean = std(x_clean);

figure;
plot(x, '-o'); hold on;
plot(x_clean, '--s');
legend('Original Data', 'Clean Data');
xlabel('Index'); ylabel('Temperature');
title('Original vs Clean Data');
grid on;

```

```

%% 2번
clear; clc;

A = [10 200 0.5;
      12 180 0.7;
      15 220 0.6;
      20 250 0.9;
      25 300 1.1];

colMin = min(A);
colMax = max(A);

A_norm = (A - colMin) ./ (colMax - colMin);

rowMean = mean(A_norm, 2);

[~, idx] = max(rowMean);

bestOriginal = A(idx, :);

```

```

bestNormalized = A_norm(idx, :);

%% 3번
clear; clc;

Channel = ["CH1";"CH2";"CH3";"CH4";"CH5";"CH6";"CH7";"CH8"];
Signal = [18.2;15.5;21.0;11.7;25.3;19.4;28.6;16.8];
Noise = [2.1;3.4;2.8;4.2;3.0;5.1;2.9;3.7];

T = table(Channel, Signal, Noise);

T.SignalToNoise = T.Signal ./ T.Noise;

meanSNR = mean(T.SignalToNoise);

T_high = T(T.SignalToNoise > meanSNR, :);

snrVec = T_high(:, 'SignalToNoise');

varSNR = var(snrVec);

%% 4번
clear; clc;

varNames = {'Class', 'Alcohol', 'MalicAcid', 'Ash', 'Alcalinity', ...
            'Magnesium', 'Phenols', 'Flavanoids', 'Nonflavanoid', ...
            'Proanthocyanins', 'ColorIntensity', 'Hue', 'OD280', 'Proline'};

Wine = readtable('wine.data', 'FileType', 'text', ...
                'ReadVariableNames', false);

Wine.Properties.VariableNames = varNames;

X = Wine(:, {'Alcohol', 'MalicAcid', 'ColorIntensity'});

X_std = zscore(X);

normVec = sqrt(sum(X_std.^2, 2));

cutoff = prctile(normVec, 90);

T_top = Wine(normVec >= cutoff, :);

T_top(:, {'Class', 'Alcohol', 'MalicAcid', 'ColorIntensity'})

%% 5번
clear; clc;

```

```

rng(1);

y = 50 + 5*randn(100,1);

y([10 25 40 70 90]) = [90 95 5 100 0];

Q1 = quantile(y, 0.25);
Q3 = quantile(y, 0.75);

IQR_val = Q3 - Q1;

lower = Q1 - 1.5*IQR_val;
upper = Q3 + 1.5*IQR_val;

idx_outlier = y < lower | y > upper;

y_clean = y(~idx_outlier);

med_clean = median(y_clean);

%% 6번
clear; clc;

Device = ["D1";"D2";"D3";"D4";"D5";"D6";"D7";"D8";"D9";"D10";"D11";"D12"];
Power = [110;130;125;160;170;150;190;200;210;180;220;240];
Throughput = [42;50;48;61;65;57;72;75;79;69;83;91];

DeviceData = table(Device, Power, Throughput);

p = DeviceData.Power;
th = DeviceData.Throughput;

R = corrcoef(p, th);
rValue = R(1,2);

if rValue >= 0.7
    disp('Strong correlation');
else
    disp('Weak correlation');
end

figure;
scatter(p, th, 'filled');
xlabel('Power');
ylabel('Throughput');
title('Power vs Throughput');
grid on;
ax = gca;

```

```

ax.FontSize = 12;
ax.LineWidth = 1.5;

%% 7번
clear; clc;

Material = ["A";"A";"A";"A";"B";"B";"B";"B";"C";"C";"C";"C"];
EnergyDensity = [245;252;238;260;310;295;305;318;180;190;176;185];

Battery = table(Material, EnergyDensity);

idxA = Battery.Material == "A";
idxB = Battery.Material == "B";
idxC = Battery.Material == "C";

meanA = mean(Battery.EnergyDensity(idxA));
meanB = mean(Battery.EnergyDensity(idxB));
meanC = mean(Battery.EnergyDensity(idxC));

stdA = std(Battery.EnergyDensity(idxA));
stdB = std(Battery.EnergyDensity(idxB));
stdC = std(Battery.EnergyDensity(idxC));

MaterialName = ["A";"B";"C"];
Mean = [meanA; meanB; meanC];
Std = [stdA; stdB; stdC];

Summary = table(MaterialName, Mean, Std);

G = groupsummary(Battery, "Material", {"mean","std"}, "EnergyDensity");

%% 8번
clear; clc;

Speed = [1.2;1.5;1.7;2.0;2.2;2.4;2.6;2.8;3.0;3.2;3.4;3.6;3.8;4.0;4.2];
Torque = [15;18;21;24;26;29;31;35;38;41;43;47;50;54;58];
SlipRate =
[0.08;0.10;0.12;0.15;0.18;0.21;0.22;0.25;0.27;0.30;0.33;0.35;0.39;0.42;0.45];
EnergyUse = [42;48;53;61;66;72;77;85;91;98;105;113;121;130;140];

Robot = table(Speed, Torque, SlipRate, EnergyUse);

X = table2array(Robot);

figure;
plotmatrix(X);

varVec = var(X);

```

```

[~, idxMax] = max(varVec);

varNames = Robot.Properties.VariableNames;
maxVarName = varNames{idxMax};

%% 9번
clear; clc;

T = readtable('train.csv');

numMissingAge = sum(ismissing(T.Age));

meanAge = mean(T.Age, 'omitnan');
T.Age(ismissing(T.Age)) = meanAge;

muFare = mean(T.Fare);
sigFare = std(T.Fare);

idxFare = abs(T.Fare - muFare) <= 3*sigFare;

T_clean = T(idxFare, :);

meanAge0 = mean(T_clean.Age(T_clean.Survived == 0));
meanAge1 = mean(T_clean.Age(T_clean.Survived == 1));

Survived = [0;1];
MeanAge = [meanAge0; meanAge1];

Result = table(Survived, MeanAge);

%% 10번
clear; clc;

Epoch = (1:8)';
LossA = [2.40;1.95;1.62;1.31;1.05;0.86;0.70;0.58];
LossB = [2.55;2.10;1.80;1.50;1.25;1.03;0.86;0.72];
LossC = [2.70;2.20;1.90;1.63;1.42;1.21;1.05;0.93];
AccA = [0.42;0.50;0.58;0.64;0.70;0.75;0.79;0.82];
AccB = [0.40;0.47;0.53;0.60;0.66;0.70;0.74;0.78];
AccC = [0.38;0.45;0.51;0.57;0.62;0.67;0.70;0.73];

TrainLog = table(Epoch, LossA, LossB, LossC, AccA, AccB, AccC);

figure;
tiledlayout(2,1);

nexttile;

```

```

plot(Epoch, LossA, '-o'); hold on;
plot(Epoch, LossB, '-s');
plot(Epoch, LossC, '-^');
legend('LossA','LossB','LossC');
xlabel('Epoch'); ylabel('Loss');
title('Loss by Epoch');
grid on;

nexttile;
plot(Epoch, AccA, '-o'); hold on;
plot(Epoch, AccB, '-s');
plot(Epoch, AccC, '-^');
legend('AccA','AccB','AccC');
xlabel('Epoch'); ylabel('Accuracy');
title('Accuracy by Epoch');
grid on;

sgtitle('Training Dashboard');

%% 11번
clear; clc;

A = [3 -1 5 -2 4;
     -4 6 -3 2 1;
       7 -5 8 -1 0;
     -2 4 -6 9 3;
       5 -3 2 -4 6];

A_clean = A;

for i = 1:size(A_clean,1)
    for j = 1:size(A_clean,2)
        if A_clean(i,j) < 0
            A_clean(i,j) = 0;
        end
    end
end

disp('Original A');
disp(A);

disp('Clean A');
disp(A_clean);

%% 12번
clear; clc;

rng(1);

```

```

t = (1:120)';

signal = 30 + 4*sin(t/8) + randn(120,1);

signal([15 45 90]) = NaN;

signal(30) = 80;
signal(75) = -20;
signal(110) = 95;

idxMissing = isnan(signal);
numMissing = sum(idxMissing);

signalFilled = signal;
fillMean = mean(signalFilled, 'omitnan');
signalFilled(idxMissing) = fillMean;

mu = mean(signalFilled);
sig = std(signalFilled);

idxOutlier = abs(signalFilled - mu) > 3*sig;

signalClean = signalFilled;
signalClean(idxOutlier) = NaN;

validSignal = signalClean(~isnan(signalClean));

normSignal = (validSignal - min(validSignal)) ./ ...
    (max(validSignal) - min(validSignal));

finalMean = mean(normSignal);
finalVar = var(normSignal);

figure;
tiledlayout(2,1);

nexttile;
plot(validSignal, '-o');
title('Valid Signal');
xlabel('Index'); ylabel('Signal');
grid on;

nexttile;
plot(normSignal, '-s');
title('Normalized Signal');
xlabel('Index'); ylabel('Normalized Value');
grid on;

```

```

%% 13번
clear; clc;

x = -5:0.05:5;
y = -5:0.05:5;

[X, Y] = meshgrid(x, y);

Z = exp(-0.08*(X.^2 + Y.^2)) .* sin(3*X) .* cos(2*Y) ...
    + 0.15*sin(X.*Y);

figure;
surf(X, Y, Z);
hold on;

shading interp;
colorbar;

xlabel('x-axis');
ylabel('y-axis');
zlabel('z=f(x,y)');
title('Detailed Surface Plot of a Multivariable Function');

grid on;
ax = gca;
ax.FontSize = 12;

view(45,30);

Z0 = zeros(size(Z));
surf(X, Y, Z0, 'FaceAlpha', 0.3, 'EdgeColor', 'none');

maxZ = max(Z(:));
minZ = min(Z(:));

[rowMax, colMax] = find(Z == maxZ, 1);
[rowMin, colMin] = find(Z == minZ, 1);

maxPoint = [X(rowMax,colMax), Y(rowMax,colMax), Z(rowMax,colMax)];
minPoint = [X(rowMin,colMin), Y(rowMin,colMin), Z(rowMin,colMin)];

plot3(maxPoint(1), maxPoint(2), maxPoint(3), 'ro', ...
    'MarkerSize', 10, 'LineWidth', 2);

plot3(minPoint(1), minPoint(2), minPoint(3), 'bx', ...
    'MarkerSize', 10, 'LineWidth', 2);

legend('Surface', 'z=0 plane', 'Maximum point', 'Minimum point');

saveas(gcf, 'complex surface.png');

```